

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

Факультет «Информатика и системы управления»
Кафедра «Теоретическая информатика и компьютерные технологии» (ИУ9)

РАСЧЁТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

по дисциплине «Алгоритмы компьютерной графики»

НА ТЕМУ:

«Программная коррекция оптических искажений в микроскопии»

Студент группы ИУ9-52Б _____ Афонькин О. О.
(Подпись, дата) (И.О. Фамилия)

Руководитель курсовой работы _____ Брагин А. В.
(Подпись, дата) (И.О. Фамилия)

Москва 2026

АННОТАЦИЯ

Работа посвящена разработке программного обеспечения для автоматической коррекции оптических искажений (хроматических аберраций) микрофотографий. Исследована природа поперечной хроматической аберрации, построена её математическая модель в виде радиального полинома, реализованы два режима оценки смещений цветовых каналов: по точечному калибровочному кадру и без калибровочного кадра (слепой). Разработан графический интерфейс с загрузкой исходного снимка и визуальным сравнением результатов «до/после». Проведено исследование зависимости времени обработки от разрешения изображения.

Объём работы: **??** с., **8** рисунков, **4** таблицы, **4** приложения, **10** источников.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

R, G, B — красный, зелёный и синий цветовые каналы (red, green, blue)

МНК — метод наименьших квадратов

Мпикс — мегапиксель

ПО — программное обеспечение

ВВЕДЕНИЕ

Оптические системы микроскопов формируют изображение в конечном спектральном диапазоне. Вследствие дисперсии показателя преломления оптических стёкол лучи различных длин волн фокусируются в разных точках, что приводит к появлению цветных ореолов и двоению контуров на изображении — хроматической аберрации [9]. Наиболее выражено это искажение на краях поля зрения. В микроскопии такое искажение не является лишь косметическим дефектом: смещение цветовых каналов на несколько пикселей искажает результаты количественных измерений размеров объектов и координат меток, что особенно критично при анализе флуоресцентных препаратов. Оптическое решение проблемы — апохроматические объективы — существенно удорожает оптическую систему, поэтому программная коррекция по оцифрованному изображению является востребованной альтернативой.

Цель работы — разработка программного обеспечения для автоматической коррекции искажений микрофотографий, вносимых оптической системой микроскопа.

Для достижения цели необходимо решить следующие **задачи**:

1) исследовать природу хроматических аберраций и построить математическую модель поперечной хроматической аберрации;
2) реализовать калибровку объектива по точечному калибровочному кадру с количественной оценкой остаточных смещений;
3) реализовать оценку смещений цветовых каналов непосредственно по самому снимку, не требующую калибровочного кадра (слепой режим);
4) разработать графический интерфейс с загрузкой исходного снимка и визуальным сравнением результатов «до/после»;
5) исследовать зависимость времени обработки изображений от их разре-

шения.

Объект исследования — цифровые изображения, содержащие хроматические аберрации. **Предмет исследования** — алгоритмы программной коррекции поперечной хроматической аберрации.

Введение, заключение, список использованных источников и приложения. В первом разделе исследуется природа аберраций, строится модель и проводится обзор существующих решений. Во втором разделе описан проект разрабатываемого программного обеспечения. Третий раздел посвящён технологии реализации. В четвёртом разделе приведены результаты экспериментального исследования.

1 Исследовательская часть

1.1 Природа хроматической аберрации

Оптическая система является диспергирующей средой: показатель преломления стекла зависит от длины волны света [1, 3]. Задача коррекции таких искажений относится к реставрации (восстановлению) изображений [2]. Вследствие этого преломляющие свойства линзы для красного и синего света различаются, и изображение точки формируется в различных положениях. Различают два вида хроматической аберрации.

Продольная (осевая) хроматическая аберрация состоит в том, что лучи разных длин волн фокусируются на разных расстояниях вдоль оптической оси. Она проявляется как размытие изображения, одинаковое по всему полю зрения, и не описывается радиальным смещением каналов.

Поперечная (латеральная) хроматическая аберрация состоит в том, что лучи разных длин волн увеличиваются линзой с различным коэффициентом. Изображение точки в разных цветовых каналах оказывается смещённым в радиальном направлении от центра поля зрения, причём в самом центре смещение равно нулю. Именно поперечная аберрация проявляется как цветные ореолы на контурах объектов, и именно она является предметом коррекции в настоящей работе.

Количественной мерой хроматической аберрации удобно считать радиальное смещение красного (R) и синего (B) каналов относительно зелёного (G): зелёный канал лежит в середине видимого диапазона и наименее подвержен искажениям. На рисунке 1 показана синтетическая точечная мишень с искусственно внесённым смещением каналов (R смещён наружу на 8 %, B внутрь на 6 %), а на рисунке 2 — увеличенный фрагмент её угла. Хорошо видны цветные ореолы, симметричные относительно центра изображения.

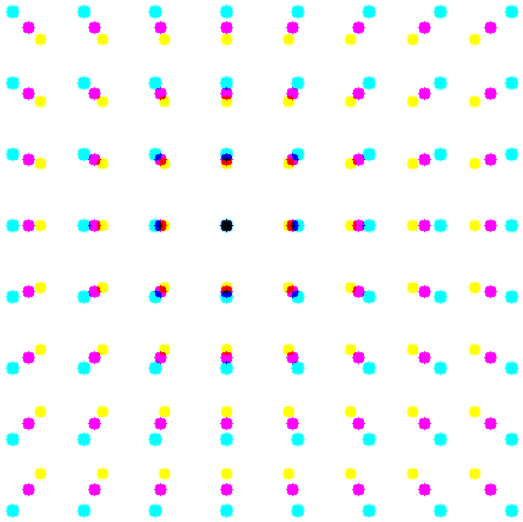


Рисунок 1 — Синтетическая точечная мишень с поперечной хроматической аберрацией (R смещён на 8 %, B — на 6 %)

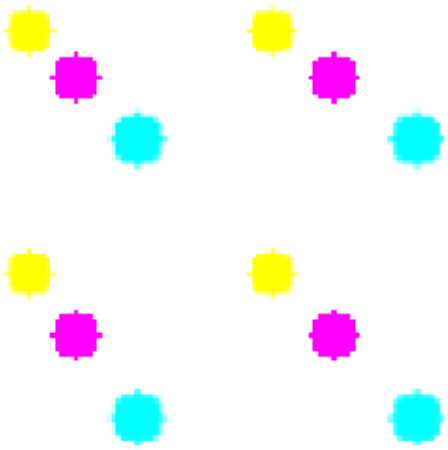


Рисунок 2 — Увеличенный фрагмент угла мишени: цветные ореолы на контурах

В микроскопии калибровку хроматических аберраций традиционно выполняют с помощью точечного препарата: микросфер (beads) или иного объекта с заведомо малым размером. Такой препарат играет роль калибровочного кадра, по которому измеряется смещение каналов [8]. Разрабатываемое ПО реализует именно такой режим калибровки, а также его слепой аналог.

1.2 Математическая модель смещения

Причиной поперечной аберрации является зависимость увеличения линзы от длины волны. Первый член разложения этой зависимости по координате даёт линейную зависимость смещения от радиуса, последующие — более высокие степени. В общем случае радиальное смещение d канала относительно опорного на расстоянии r от оптического центра описывается полиномом:

$$d(r) = c_1 r + c_2 r^2 + c_3 r^3, \tag{1}$$

где c_1, c_2, c_3 — коэффициенты модели, подлежащие определению при калибровке [5, 7].

В слепом режиме, когда калибровочный кадр отсутствует и смещения оцениваются по краям самого снимка, используется линейная модель:

$$d(r) = kr, \tag{2}$$

где k — коэффициент пропорциональности [10]. Линейная модель грубее, однако полином высокого порядка, построенный по зашумлённым краям реального снимка, при экстраполяции даёт неустойчивый результат; кроме того, как показано в разделе 1.1, линейный член является доминирующим.

Измерение смещения канала по калибровочному кадру выполняется следующим образом. Для каждой точки калибровочной мишени определяется её положение в каждом цветовом канале, после чего вычисляется проекция вектора смещения на радиальное направление:

$$d_i = (\mathbf{p}_{c,i} - \mathbf{p}_{g,i}) \cdot \frac{\mathbf{p}_{g,i} - \mathbf{p}_0}{\|\mathbf{p}_{g,i} - \mathbf{p}_0\|}, \tag{3}$$

где $\mathbf{p}_{c,i}$ — положение i-й точки в измеряемом канале, $\mathbf{p}_{g,i}$ — её положение в опорном (зелёном) канале, $\mathbf{p}_0$ — оптический центр. Тем самым задача сводится к скалярным данным (r_i, d_i).

Коэффициенты модели (**??**) находятся методом наименьших квадратов: минимизируется сумма квадратов невязок

$$S = \sum_i \Big(d_i - \sum_{j=1}^{3} c_j r_i^j \Big)^2 \;\to\; \min_{c_1, c_2, c_3}. \tag{4}$$

Задача (**??**) линейна относительно коэффициентов и решается через нормальные уравнения.

Для оценки смещения по краям снимка (слепой режим) используются профили яркости вдоль радиального направления. Смещение измеряется как аргумент максимума нормированной взаимной корреляции профиля измеряемого канала с профилем опорного:

$$\rho(s) = \frac{\sum_t \big(g(t) - \bar{g}\big)\big(c(t+s) - \bar{c}_s\big)}{\sqrt{\sum_t \big(g(t) - \bar{g}\big)^2 \sum_t \big(c(t+s) - \bar{c}_s\big)^2}}, \tag{5}$$

где $g(t)$ и $c(t)$ — отсчёты профилей опорного и измеряемого каналов, $\bar{g}$ и $\bar{c}_s$ — их средние значения. Максимум дискретной корреляции уточняется до субпиксельной точности параболической интерполяцией трёх соседних отсчётов:

$$\hat{s} = s^* + \frac{1}{2} \cdot \frac{y_- - y_+}{y_- - 2M + y_+}, \tag{6}$$

где M — значение корреляции в максимуме s^*, y_- и y_+ — значения корреляции в соседних отсчётах слева и справа.

1.3 Обзор существующих решений

Задача коррекции хроматических аберраций решается несколькими классами программных средств. Сравнение представлено в таблице 1.

Таблица 1 — Обзор существующих средств коррекции хроматических аберраций

Средство	Режим коррекции	Модель смещения
Adobe Lightroom Classic	профиль объектива или ручная правка	попиксельные ползунки, неявная модель по полю кадра
OpenCV 5.0	по калибровочному кадру мишени	полином высокой степени (до 11-й), оценивается МНК [4]
Hugin (fulla)	по контрольным точкам	полиномиальная модель по радиусу
Разрабатываемое ПО	по калибровочному кадру и слепой (по снимку)	кубический полином (**??**) и линейная модель (**??**)

Фотографические редакторы (Lightroom, RawTherapee) применяют профили объективов либо ручную коррекцию и ориентированы на интерактивную работу, а не на автоматическую пакетную обработку микрофотографий. Библиотека OpenCV предоставляет функцию `correctChromaticAberration`, основанную на калибровке по кадру мишени с полиномиальной моделью высокой степени [4]; это решение универсально, но требует отдельного калибровочного кадра и не имеет слепого режима. Разрабатываемое ПО сочетает оба режима и ориентировано на работу с одиночным снимком в отсутствие калибровки.

1.4 Выбор средств реализации

В качестве языка реализации выбран Python 3 ввиду наличия зрелых библиотек численных вычислений и обработки изображений. Для работы с массивами используется NumPy [6], для операций над изображениями (свёртки, ремап, цветовые преобразования) — OpenCV [4]. Графический интерфейс реализован средствами штатной библиотеки Tkinter, не требующей дополнительных зависимостей. Модульное тестирование выполняется с помощью

библиотеки pytest. Перечисленный набор непротиворечив, свободно распространяется и полностью покрывает задачи работы.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемому ПО предъявляются следующие требования:

1) автоматическая коррекция поперечной хроматической аберрации без ручного вмешательства пользователя;
2) возможность работы как по калибровочному кадру точечной мишени, так и без него (слепой режим);
3) количественная оценка внесённой поправки и остаточного смещения каналов;
4) загрузка исходного снимка через графический интерфейс и визуальное сравнение результатов «до» и «после»;
5) сохранение результатов обработки на диск независимо от состояния интерфейса;
6) работа с изображениями широкого диапазона разрешений (от 0,3 до 12 Мпикс) без существенного роста времени обработки.

2.2 Архитектура программной системы

Система построена по модульному принципу и разделена на вычислительное ядро (пакет `achrom`) и графический интерфейс (`app.py`). Ядро не зависит от интерфейса и может использоваться как библиотека. Архитектура показана на рисунке 3.

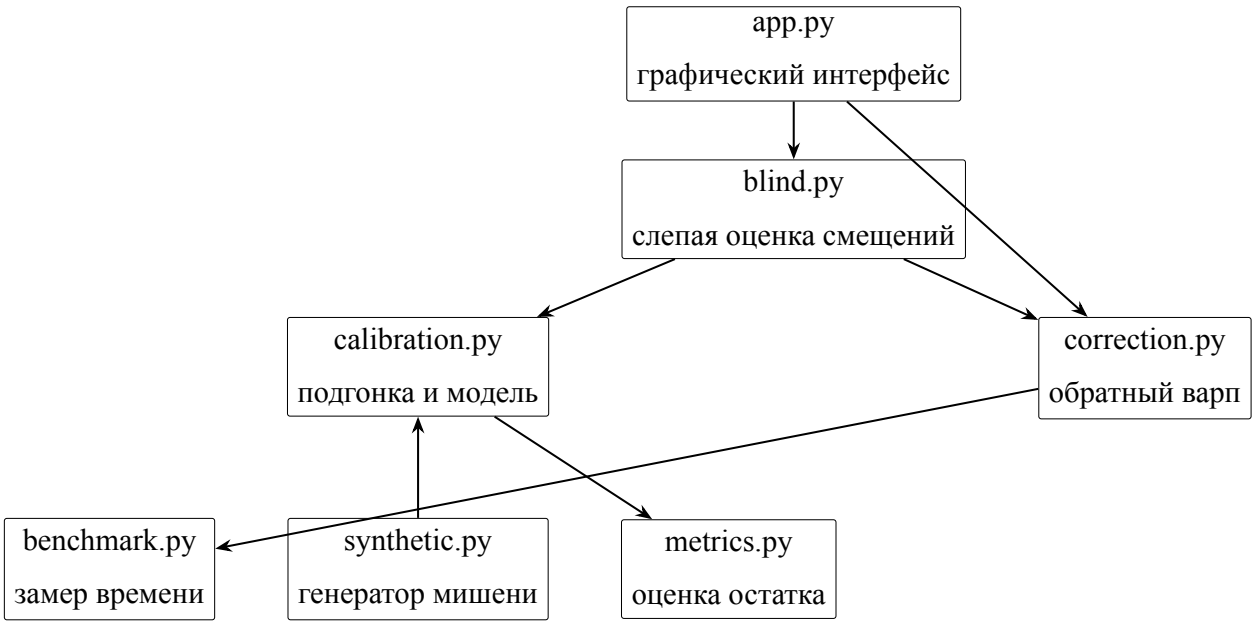


Рисунок 3 — Архитектура программной системы

Модуль `calibration.py` выполняет обнаружение точек мишени, измерение смещений (**??**) и подгонку модели (**??**) методом наименьших квадратов, а также содержит функцию вычисления полинома по коэффициентам. Модуль `correction.py` реализует обратное радиальное смещение каналов. Модуль `blind.py` реализует слепую оценку смещений по краям снимка. Вспомогательные модули `synthetic.py`, `metrics.py` и `benchmark.py` формируют тестовые данные, оценивают остаточное смещение и измеряют время обработки соответственно.

2.3 Калибровка объектива по калибровочному кадру

Калибровка по кадру точечной мишени включает следующие этапы:

1) обнаружение точек (дисков) мишени в каждом цветовом канале;
2) определение положения каждой точки с субпиксельной точностью как центра масс окрестности;
3) сопоставление точек между каналами и вычисление радиальных сме-

щений по формуле (**??**);

4) подгонка коэффициентов модели (**??**) методом наименьших квадратов.

Обнаружение дисков работает на обоих типах мишени (тёмные точки на светлом фоне и светлые на тёмном) благодаря анализу знака локального контраста. Качество коррекции количественно оценивается средним и максимальным остаточным смещением каналов после её применения.

Проверка корректности калибровки выполнена на синтетическом кадре с заведомо известными коэффициентами смещения (R +8 %, B −6 %). После калибровки такие же коэффициенты были применены к другому кадру с той же аберрацией, но иным расположением точек. Результат показан на рисунке 4: среднее расхождение каналов уменьшилось с 11,92 до 0,83 пикс (максимальное — с 24,88 до 1,80 пикс), то есть более чем на порядок.



Рисунок 4 — Результат калиброванной коррекции синтетического кадра: слева — до коррекции, справа — после

2.4 Алгоритм обратной коррекции

Коррекция состоит в обратном радиальном смещении красного и синего каналов к опорному зелёному. Для каждого выходного пикселя на радиусе

r требуется найти такой радиус s во входном изображении, значение с которого искажено меньше всего. Поскольку смещение задано в терминах входного радиуса, искомый радиус s является решением уравнения фиксированной точки

$$s = r + d(s), \tag{7}$$

которое находится методом последовательных приближений:

$$s^{(0)} = r, \qquad s^{(n+1)} = r + d\left(s^{(n)}\right). \tag{8}$$

Для используемых моделей смещения уравнение (**??**) решается итерациями (**??**), которые сходятся за несколько шагов. Пересчёт координат выполняется одним вызовом функции ремапа для канала с билинейной интерполяцией; за границами кадра значения продлеваются краевым пикселем, что предотвращает появление чёрной рамки.

2.5 Слепой режим оценки смещений

Слепой режим не требует калибровочного кадра и опирается на тот факт, что контрастный край снимка присутствует во всех цветовых каналах одинаково, а поперечная аберрация лишь растягивает каналы радиально. Алгоритм состоит из следующих шагов:

1) по градиенту яркости зелёного канала отбираются точки краёв (верхние 2 % по величине градиента), при этом исключаются точки вблизи центра и на границе кадра, а также точки, край которых не перпендикулярен радиальному направлению;
2) через каждую отобранную точку строится профиль яркости вдоль радиуса (окно ±28 пикс) для каждого канала;
3) смещение измеряется как аргумент максимума корреляции (**??**) с субпиксельным уточнением (**??**); пары с низкой корреляцией ($< 0{,}75$) от-

брасываются;

4) по данным (r_i, d_i) подгоняется линейная модель (**??**); выбросы с остатком более 1 пикс отбрасываются, подгонка повторяется;
5) найденные коэффициенты применяются функцией коррекции, после чего оценка повторяется по скорректированному изображению. Итерации прекращаются, когда остаточное смещение становится менее 0,5 пикс (но не более трёх итераций).

Итеративная схема необходима потому, что смещение измеряется по уже отчасти исправленному изображению: каждая следующая итерация уточняет коэффициенты и уменьшает остаток.

2.6 Графический интерфейс

Интерфейс реализует основной сценарий работы: пользователь нажимает кнопку «Загрузить снимок», выбирает файл изображения, после чего выполняется слепая автокоррекция. Обработка идёт в фоновом потоке, поэтому окно интерфейса не замораживается. Обновление элементов интерфейса производится исключительно из главного потока опросом состояния задачи через механизм отложенных вызовов Tkinter, поскольку библиотека Tkinter не допускает обращений из других потоков.

По завершении обработки в подкаталог `results` записываются три файла: изображение до коррекции, изображение после коррекции и парное изображение для сравнения (увеличенные фрагменты наиболее удалённого от центра угла кадра, где аберрация максимальна). Файлы сохраняются на диск до отображения, поэтому результат не теряется даже при закрытии окна во время обработки. Парное изображение открывается в системном просмотрщике изображений. Все исключения протоколируются в журнале `app.log`.

Режим калибровки по точечному кадру реализован как программный интерфейс библиотеки и продемонстрирован в разделе 4; графический интерфейс ориентирован на типовой сценарий обработки одиночного снимка.

3 Технологическая часть

3.1 Технологический стек

Программное обеспечение реализовано на языке Python 3.9. Для численных вычислений используются библиотеки NumPy 2.0 и OpenCV 5.0, для работы с изображениями при сохранении — OpenCV и Pillow. Графический интерфейс построен на штатной библиотеке Tkinter. Тестирование выполняется библиотекой pytest. Все зависимости свободно распространяются.

3.2 Организация исходного кода

Исходный код разделён на пакет вычислительного ядра `achrom` и головной модуль `app.py`. Пакет содержит модули `calibration`, `correction`, `blind`, `synthetic`, `metrics`, `benchmark` в соответствии с архитектурой (рисунок **??**). Полные листинги модулей приведены в приложениях А–Г.

3.3 Модульное тестирование

Разработка велась по методике тестирования на основе спецификации: для каждой функции сначала создавался тест, затем реализация. Ожидаемые значения в тестах берутся из независимого источника — известных коэффициентов генератора синтетических изображений или ручного расчёта, а не из повторного применения проверяемого алгоритма. Всего реализовано 13 тестов, покрывающих:

— обнаружение точек мишени на светлом и тёмном фоне;

— измерение смещений на кадре с известным коэффициентом;
— восстановление полиномиальной модели по синтетическим данным;
— коррекцию кадра, не входившего в обучающую выборку;
— слепую оценку смещения на синтетическом кадре с известным коэффициентом;
— отсутствие ложной коррекции при нулевой аберрации;
— работу интерфейса коррекции во всём диапазоне разрешений и замер времени.

4 Аналитическая часть

4.1 Тестовые данные

Исследование проводилось на синтетических кадрах с известной аберрацией и на реальных снимках, содержащих выраженную поперечную хроматическую аберрацию. Поскольку открытые микрофотографии с известными параметрами аберрации в свободном доступе отсутствуют, в качестве реальных данных использованы снимки с аналогичным характером искажений: макросъёмка и кадры, снятые объективами без хроматической коррекции. Синтетические кадры имитируют калибровочный препарат микроскопа. Характеристики тестовых изображений приведены в таблице 2.

Таблица 2 — Характеристики тестовых изображений

Файл	Описание	Разрешение	Смещение R / B, первый проход
real_macro_ca.jpg	макросъёмка, Canon PowerShot	5184×2912	−8,7 / +10,9
lens_ca.jpg	кадр с аберрацией объектива	2462×2532	−8,1 / +14,0
holga_tatra.jpg	съёмка камерой Holga	3840×2566	−2,2 / +2,7
detail_ca.jpg	кадр с умеренной аберрацией	2592×2592	−1,0 / +2,1
real_blende8.jpg	контрольный кадр без аберрации	2000×1328	−0,1 / +0,2

Изображения заимствованы из открытого архива Wikimedia Commons под свободными лицензиями (CC0, CC BY-SA 3.0, CC BY-SA 4.0). Последнее изображение является контрольным: оно снято камерой с внутрикамерной коррекцией и не содержит заметной аберрации.

4.2 Результаты коррекции

Слепая коррекция выполнена для каждого изображения. Суммарная внесённая поправка, остаточное смещение (по результатам финальной оценки) и число выполненных итераций приведены в таблице 3.

Таблица 3 — Результаты слепой коррекции

Изображение	Суммарная поправка R / B, пикс	Остаток R / B, пикс	Итераций	Время, с
real_macro_ca.jpg	−9,7 / +15,6	−0,4 / +0,7	3	2,0
lens_ca.jpg	−8,2 / +14,7	−0,0 / +0,1	2	0,7
holga_tatra.jpg	−2,2 / +2,7	−0,1 / +0,2	1	0,6
detail_ca.jpg	−1,0 / +2,1	−0,3 / +0,3	1	0,3
real_blende8.jpg	—	—	0	<0,1

Во всех случаях с выраженной аберрацией остаточное смещение после коррекции снизилось до долей пикселя (менее 0,7 пикс), что соответствует погрешности измерения. Для контрольного изображения без аберрации оценённое смещение не превысило порога 0,5 пикс, и коррекция не выполнялась — это подтверждает отсутствие ложных срабатываний.

На рисунке 5 показан результат для макросъёмки с наибольшим смещением, на рисунке 6 — для кадра с аберрацией объектива. На обоих фрагментах видно устранение цветных ореолов на контурах.

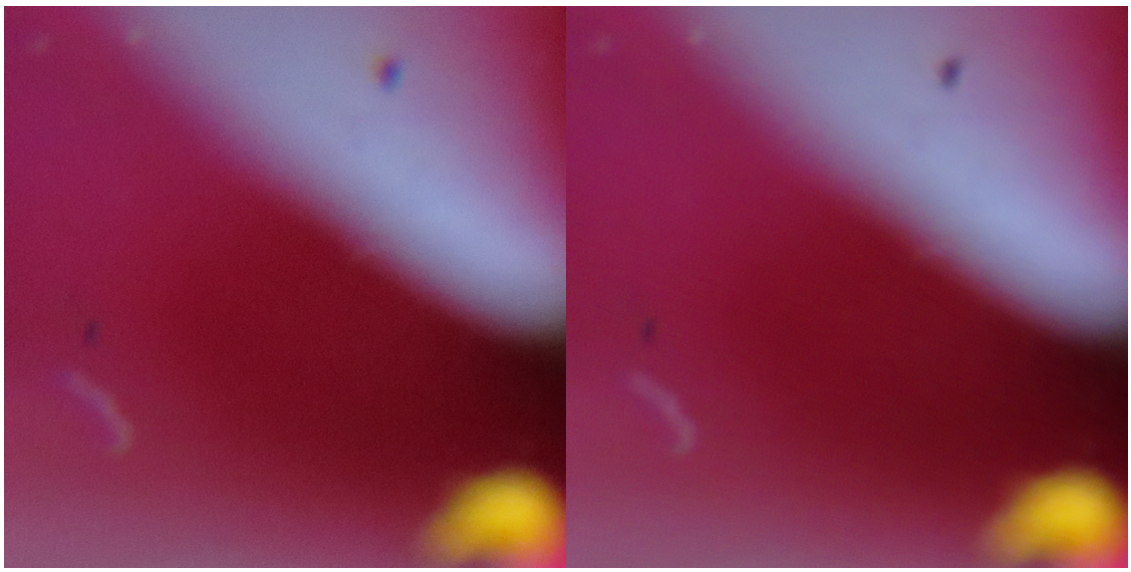


Рисунок 5 — Макросъёмка с выраженной поперечной аберрацией (R −9,7 пикс, B +15,6 пикс): слева — до, справа — после

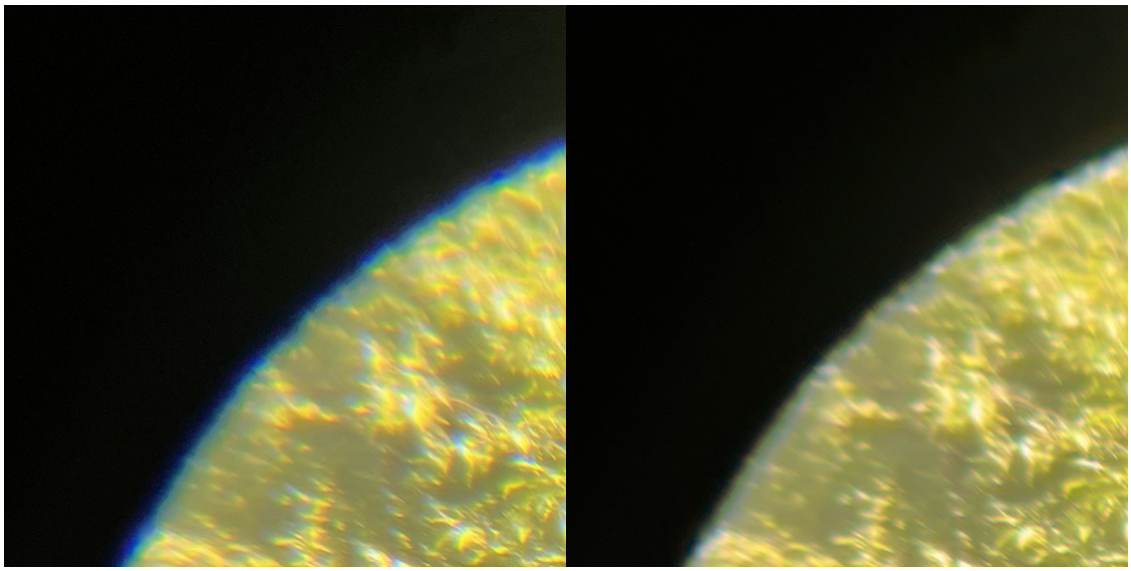


Рисунок 6 — Кадр с хроматической аберрацией объектива (R −8,1 пикс, B +14,0 пикс): слева — до, справа — после

Отдельно исследован случай продольной хроматической аберрации, при которой цветовые каналы различаются степенью размытия, а не радиальным смещением. Слепой алгоритм в этом случае не обнаруживает значимого смещения и не вносит коррекции, что согласуется с теоретической моделью: продольная аберрация не описывается радиальным растяжением каналов и

требует других методов (например, деконволюции с разными ядрами для каналов).

4.3 Зависимость времени обработки от разрешения

Время коррекции измерено для изображений от 0,3 до 12 Мпикс. Результаты приведены в таблице 4 и на рисунке 8.

Таблица 4 — Время коррекции в зависимости от разрешения

Разрешение	Пикселей, Мпикс	Время, мс	Удельно, мс/Мпикс
640×480	0,31	24,5	79,8
1280×960	1,23	64,8	52,7
1920×1440	2,76	147,3	53,4
2560×1920	4,92	254,4	51,7
3840×2880	11,06	614,1	55,5

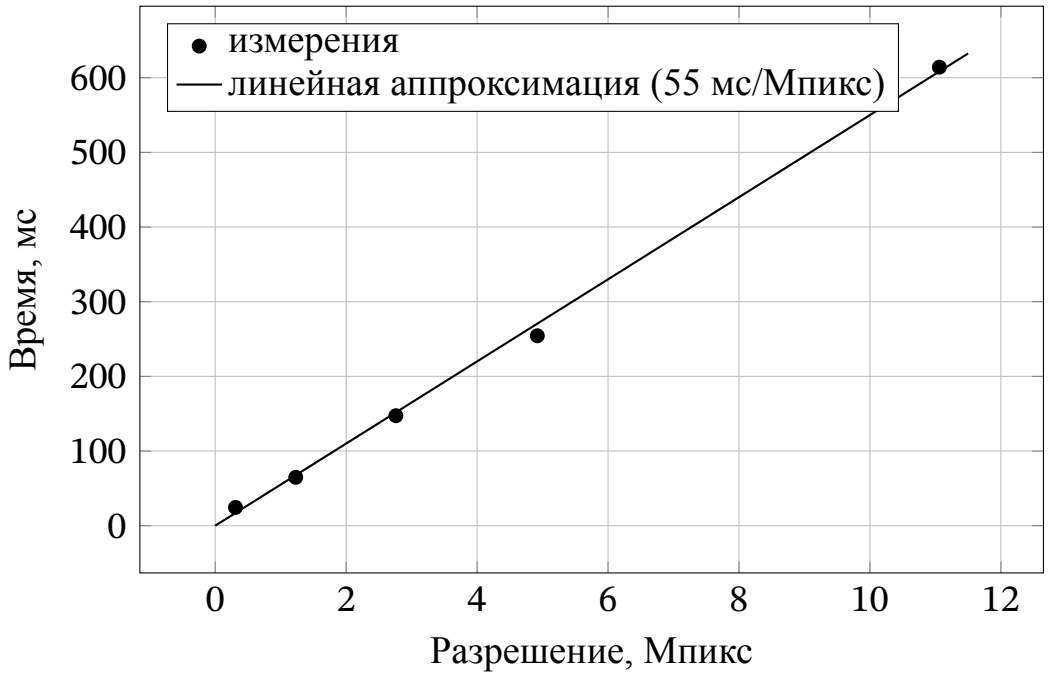


Рисунок 7 — Зависимость времени коррекции от разрешения изображения

Время обработки возрастает практически линейно с числом пикселей;

удельные затраты составляют около 55 мс на мегапиксель. Наименьшее разрешение имеет повышенные удельные затраты (80 мс/Мпикс) из-за постоянных накладных расходов, которые при больших разрешениях становятся пренебрежимо малы. Таким образом, время обработки снимка типовой микрофотографии (несколько мегапикселей) составляет доли секунды, что приемлемо для практического применения.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы разработано программное обеспечение для автоматической коррекции поперечной хроматической аберрации микрофотографий. Получены следующие результаты.

Исследована природа хроматических аберраций и построена математическая модель поперечной аберрации в виде радиального полинома, учитывающая доминирующий линейный член и нелинейные поправки.

Реализована калибровка объектива по точечному калибровочному кадру с количественной оценкой остаточных смещений. На синтетическом кадре с известной аберрацией среднее расхождение каналов уменьшилось с 11,92 до 0,83 пикс (более чем на порядок).

Реализован слепой режим оценки смещений по самому снимку, не требующий калибровочного кадра. На реальных изображениях с аберрацией остаточное смещение после коррекции снизилось до долей пикселя; при этом алгоритм не вносит ложной коррекции при отсутствии аберрации.

Разработан графический интерфейс с загрузкой снимка и визуальным сравнением результатов «до/после»; обработка идёт в фоновом потоке, результаты сохраняются на диск независимо от состояния окна.

Исследована зависимость времени обработки от разрешения: время растёт линейно и составляет около 55 мс на мегапиксель.

В качестве ограничений отмечено, что продольная хроматическая аберрация не описывается радиальной моделью смещения и не корректируется разработанными средствами. Направлениями дальнейшей работы являются коррекция продольной аберрации методами деконволюции, автоматическое определение оптического центра и распараллеливание вычислений на графическом процессоре.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) 1. Гонсалес Р., Вудс Р. Цифровая обработка изображений. — 3-е изд., испр. и доп. — М.: Техносфера, 2012. — 1104 с.
2) Прэтт У. Цифровая обработка изображений: в 2 кн. — М.: Мир, 1982. — Кн. 1. — 312 с.; кн. 2. — 480 с.
3) Яне Б. Цифровая обработка изображений. — М.: Техносфера, 2007. — 584 с.
4) Bradski G. The OpenCV Library // Dr. Dobb's Journal of Software Tools. — 2000. — Vol. 25, no. 11. — P. 120–125.
5) Brown D. C. Decentering distortion of lenses // Photogrammetric Engineering. — 1966. — Vol. 32, no. 3. — P. 444–462.
6) Harris C. R. et al. Array programming with NumPy // Nature. — 2020. — Vol. 585. — P. 357–362.
7) Hartley R., Zisserman A. Multiple View Geometry in Computer Vision. — 2nd ed. — Cambridge : Cambridge University Press, 2004. — 655 p.
8) Leiwe M. N., Fujimoto S., Imai T. Post hoc correction of chromatic aberrations in large-scale volumetric images in confocal microscopy // Frontiers in Neuroanatomy. — 2021. — Vol. 15. — Art. 760063.
9) Szeliski R. Computer Vision: Algorithms and Applications. — 2nd ed. — Cham: Springer, 2022. — 925 p.
10) Zhang Z. A Flexible New Technique for Camera Calibration // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2000. — Vol. 22, no. 11. — P. 1330–1334.

ПРИЛОЖЕНИЕ А

Модули калибровки и формирования тестовых данных

Листинг А.1 — Модуль calibration.py

```
import cv2
import numpy as np


POLY_DEGREE = 3  # порядок модели хроматического смещения d(r)



def detect_dots(gray):
    """Центроиды точек калибровочного кадра, отсортированные по (
   x, y).

    Работает и с тёмными точками на светлом фоне, и наоборот.
    """
    if gray.mean() > 127:
        gray = 255 - gray
    _, labels, _, centroids = cv2.connectedComponentsWithStats(
        (gray > 128).astype(np.uint8), connectivity=8
    )
    return centroids[1:]  # [0] — фон



def measure_displacements(frame_bgr, center):
    """Смещения R и B каналов относительно G опорного() по
```

```
    каждой точке.

     Возвращает (r, dr_red, dr_blue): радиус точки от оптического
    центра и
     проекции смещений на радиальное направление, в пикселях.
     """
     b, g, r_ch = cv2.split(frame_bgr)
     pts_g = detect_dots(g)
     pts_r = detect_dots(r_ch)
     pts_b = detect_dots(b)

     cx, cy = center
     matched_r = _nearest(pts_g, pts_r)
     matched_b = _nearest(pts_g, pts_b)

     r = np.hypot(pts_g[:, 0] - cx, pts_g[:, 1] - cy)
     dr_red = _radial_component(pts_g, matched_r, cx, cy, r)
     dr_blue = _radial_component(pts_g, matched_b, cx, cy, r)
     return r, dr_red, dr_blue


def radial_model(coeffs, r):
     """d(r) по коэффициентам [c1, c2, ...]: d = c1*r + c2*r^2 +
    ..."""
     return sum(c * np.asarray(r) ** i for i, c in enumerate(
    coeffs, start=1))
```

```
def fit_radial(r, d, degree=POLY_DEGREE):
    """МНКподгонка- полинома d(r) без свободного члена в( центре
   d=0)."""
    r = np.asarray(r, float)
    A = np.stack([r ** i for i in range(1, degree + 1)], axis=1)
    return np.linalg.lstsq(A, d, rcond=None)[0]


def _nearest(ref, candidates):
    """Для каждой точки ref — ближайшая из candidates."""
    d = np.linalg.norm(ref[:, None] - candidates[None], axis=2)
    return candidates[d.argmin(axis=1)]


def _radial_component(ref, shifted, cx, cy, r):
    disp = shifted - ref
    radial = np.stack([ref[:, 0] - cx, ref[:, 1] - cy], axis=1)
    safe_r = np.where(r == 0, 1.0, r)
    proj = (disp * radial).sum(axis=1) / safe_r
    return np.where(r == 0, 0.0, proj)  # на оптической оси
   сдвиг не определён
```

Листинг А.2 — Модуль synthetic.py

```
import cv2
import numpy as np
```

```
def apply_radial_shift(channel, center, k):
    """Смещает каждый пиксель наружу от центра на d(r) = k*r.

    Возвращает новый канал того же размера (uint8).
    """
    h, w = channel.shape
    cx, cy = center
    ys, xs = np.mgrid[0:h, 0:w].astype(np.float32)
    scale = 1.0 / (1.0 + k)
    map_x = cx + (xs - cx) * scale
    map_y = cy + (ys - cy) * scale
    return cv2.remap(channel, map_x, map_y, cv2.INTER_LINEAR,
                     borderMode=cv2.BORDER_REPLICATE)


def make_dot_frame(shape=(300, 300), offset=50, step=50, k_red
   =0.0, k_blue=0.0,
                   radius=3):
    """Синтетический калибровочный кадр: чёрные точки на белом,
    R и B каналы смещены радиально модель( d(r) = k*r).

    Возвращает кадр( BGR, оптический центр).
    """
    base = np.full(shape, 255, np.uint8)
    for y in range(offset, shape[0], step):
```

```
        for x in range(offset, shape[1], step):
            cv2.circle(base, (x, y), radius, 0, -1)
    center = (shape[1] / 2, shape[0] / 2)
    frame = cv2.merge([
        apply_radial_shift(base, center, k_blue),
        base.copy(),
        apply_radial_shift(base, center, k_red),
    ])  # OpenCV: B, G, R
    return frame, center
```

Листинг А.3 — Модуль metrics.py

```
import numpy as np

from .calibration import measure_displacements


def channel_residual(frame_bgr, center):
    """Насколько R и B разошлись с G: среднее(, максимум) в
   пикселях."""
    r, dr_red, dr_blue = measure_displacements(frame_bgr, center)
    d = np.abs(np.concatenate([dr_red, dr_blue]))
    return d.mean(), d.max()
```

ПРИЛОЖЕНИЕ Б

Модули коррекции и замера времени

Листинг Б.4 — Модуль correction.py

```
import cv2
import numpy as np

from .calibration import radial_model



def correct(frame_bgr, center, coeffs_red, coeffs_blue):
    """Возвращает кадр, где R и B варпнуты обратно к опорному G."
   ""
    b, g, r = cv2.split(frame_bgr)
    return cv2.merge([
        _warp_back(b, center, coeffs_blue),
        g,
        _warp_back(r, center, coeffs_red),
    ])



def _warp_back(channel, center, coeffs):
    """Обратное радиальное смещение: выходной пиксель на радиусе
   r
    берёт значение с радиуса s, где s - d(s) = r фиксированная(
   точка).
```

```
    """
    h, w = channel.shape
    cx, cy = center
    ys, xs = np.mgrid[0:h, 0:w].astype(np.float32)
    dx, dy = xs - cx, ys - cy
    r = np.hypot(dx, dy)

    s = r.copy()
    for _ in range(3):
        s = r + radial_model(coeffs, s)

    scale = np.where(r == 0, 1.0, s / np.where(r == 0, 1.0, r))
    map_x = (cx + dx * scale).astype(np.float32)
    map_y = (cy + dy * scale).astype(np.float32)
    return cv2.remap(channel, map_x, map_y, cv2.INTER_LINEAR,
                     borderMode=cv2.BORDER_REPLICATE)
```

Листинг Б.5 — Модуль benchmark.py

```
import time

import numpy as np

from .correction import correct


def benchmark(coeffs_red, coeffs_blue, sizes, center=None):
```

```
    """Время коррекции мс() для каждого разрешения. → [(w, h, ms)
, ...]"""
    rows = []
    for w, h in sizes:
        rng = np.random.default_rng(0)
        frame = rng.integers(0, 255, (h, w, 3), dtype=np.uint8)
        cx, cy = center or (w / 2, h / 2)
        t0 = time.perf_counter()
        correct(frame, (cx, cy), coeffs_red, coeffs_blue)
        rows.append((w, h, (time.perf_counter() - t0) * 1000))
    return rows
```

ПРИЛОЖЕНИЕ В

Модуль слепой оценки смещений

Листинг В.6 — Модуль blind.py

```
"""Слепая оценка хроматических смещений без калибровочного кадра.
   Контрастный

 край снимка одинаков во всех цветовых каналах, поэтомусмещение
 канала вдоль радиуса измеряется непосредственно по краямсамого
 изображения.
"""
import cv2
import numpy as np

from .calibration import fit_radial, radial_model

HALF = 28   # полуокно профиля вдоль радиуса, px
S_MAX = 24  # предел поиска сдвига, px
MIN_R = 20  # ближе к центру смещение мало и шумит — пропускаем
NCC_THR = 0.75
MAX_POINTS = 500


def estimate_displacements_blind(frame_bgr, center, degree=1):
    """(r, dr_red, dr_blue) по краям самого снимка — тот же
   контракт,
```

```
что у measure_displacements по калибровочному кадру.

По умолчанию линейная модель d(r)=k*r: латеральная CA
физически
близка к радиальному масштабированию канала, а полином высших
степеней на шумных краях реальных фото экстраполирует дико.
"""
b, g, r_ch = [c.astype(np.float32) for c in cv2.split(
frame_bgr)]
h, w = g.shape
cx, cy = center

pts = _edge_points(g, cx, cy, h, w)
if len(pts) == 0:
    return np.array([]), np.array([]), np.array([])

# линии вдоль радиуса через каждую точку: один remap на канал
t = np.arange(-HALF, HALF + 1, dtype=np.float32)
d = pts - np.array([cx, cy], np.float32)
rad = np.linalg.norm(d, axis=1, keepdims=True)
u = d / rad
lines = pts[:, None, :] + t[None, :, None] * u[:, None, :]
map_x = lines[:, :, 0]
map_y = lines[:, :, 1]
pg = cv2.remap(g, map_x, map_y, cv2.INTER_LINEAR)
pr = cv2.remap(r_ch, map_x, map_y, cv2.INTER_LINEAR)
pb = cv2.remap(b, map_x, map_y, cv2.INTER_LINEAR)
```

```
    r = rad[:, 0]
    dr_red, ok_r = _shift_along_profiles(pg, pr)
    dr_blue, ok_b = _shift_along_profiles(pg, pb)
    keep = ok_r & ok_b
    r, dr_red, dr_blue = r[keep], dr_red[keep], dr_blue[keep]
    if len(r) >= 10:
        inliers = (_inliers(r, dr_red, degree)
                   & _inliers(r, dr_blue, degree))
        if inliers.sum() >= 10:
            r, dr_red, dr_blue = r[inliers], dr_red[inliers],
  dr_blue[inliers]
    return r, dr_red, dr_blue


def _edge_points(g, cx, cy, h, w):
    gx = cv2.Sobel(g, cv2.CV_32F, 1, 0)
    gy = cv2.Sobel(g, cv2.CV_32F, 0, 1)
    mag = cv2.magnitude(gx, gy)
    ys, xs = np.where(mag > np.percentile(mag, 98))
    m = 2 * HALF + 2
    x = xs.astype(np.float32) - cx
    y = ys.astype(np.float32) - cy
    r = np.hypot(x, y)
    ok = (
        (xs > m) & (xs < w - m) & (ys > m) & (ys < h - m) & (r >
  MIN_R)
```

```
    )
    # край должен быть перпендикулярен радиусу: иначе радиальная
линия
    # режет его по касательной и профиль не измеряет сдвиг
    gxv, gyv = gx[ys, xs], gy[ys, xs]
    cos_with_radial = np.abs(
        (gxv * x + gyv * y) / (np.hypot(gxv, gyv) * r + 1e-9)
    )
    ok &= cos_with_radial > 0.7
    pts = np.stack([xs[ok], ys[ok]], axis=1).astype(np.float32)
    if len(pts) > MAX_POINTS:
        sel = np.random.default_rng(0).choice(len(pts),
MAX_POINTS, False)
        pts = pts[sel]
    return pts


def _shift_along_profiles(pg, pc):
    """Субпиксельный сдвиг pc относительно pg построчно (NCC +
параболa).

    Возвращает (dr, ok): dr — сдвиг вдоль радиуса, ok — строки с
уверенным пиком.
    """
    shifts = np.arange(-S_MAX, S_MAX + 1)
    scores = np.empty((pg.shape[0], len(shifts)), np.float32)
    for i, s in enumerate(shifts):
```

```
        if s >= 0:
            scores[:, i] = _ncc(pg[:, : pg.shape[1] - s], pc[:, s
    :])
        else:
            scores[:, i] = _ncc(pg[:, -s:], pc[:, : pc.shape[1] +
     s])

     peak = scores.argmax(axis=1)
     ok = np.take_along_axis(scores, peak[:, None], 1)[:, 0] >
    NCC_THR

     dr = shifts[peak].astype(np.float64)
     left = np.take_along_axis(scores, np.clip(peak - 1, 0, len(
    shifts) - 1)[:, None], 1)[:, 0]
     right = np.take_along_axis(scores, np.clip(peak + 1, 0, len(
    shifts) - 1)[:, None], 1)[:, 0]
     mid = np.take_along_axis(scores, peak[:, None], 1)[:, 0]
     denom = left - 2 * mid + right
     refine = np.where(denom != 0, 0.5 * (left - right) / np.where
    (denom != 0, denom, 1.0), 0.0)
     return dr + np.clip(refine, -1, 1), ok


def _ncc(a, b):
     a = a - a.mean(axis=1, keepdims=True)
     b = b - b.mean(axis=1, keepdims=True)
     num = (a * b).sum(axis=1)
```

```
    den = np.sqrt((a * a).sum(axis=1) * (b * b).sum(axis=1)) + 1e
-9
    return num / den


def _inliers(r, dr, degree):
    """Маска строк, попадающих в модель с остатком < 1px."""
    resid = np.abs(dr - radial_model(fit_radial(r, dr, degree), r
))
    return resid < 1.0
```

ПРИЛОЖЕНИЕ Г

Графический интерфейс

Листинг Г.7 — Модуль app.py

```
"""Графический интерфейс программы коррекции хроматических
   аберраций.Обработка

 выполняется в фоновом потоке, а обновление интерфейса —только
 из главного потока поллингом через root.after,
   посколькубиблиотека
 tkinter не допускает обращений из других потоков.Результаты
 сохраняются на диск в папку results/ и открываютсяв
 системном просмотрщике изображений.Исключения

 обработчиков кнопок протоколируются в журнале app.log.
"""
import functools
import logging
import os
import subprocess
import threading
import time

os.environ.setdefault("TK_SILENCE_DEPRECATION", "1")

import tkinter as tk
```

```
from tkinter import filedialog, messagebox


import cv2
import numpy as np


from achrom.blind import estimate_displacements_blind
from achrom.calibration import fit_radial, radial_model
from achrom.correction import correct


THUMB_W = 560  # ширина каждой половины имя<>_diff.png углы(
   допосле/)


logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    handlers=[
        logging.FileHandler("app.log", encoding="utf-8"),
        logging.StreamHandler(),
    ],
)
log = logging.getLogger("achrom.app")



def log_errors(fn):
    """Обёртка для кнопок: исключение — трейсбек в лог +
   сообщение юзеру."""
    @functools.wraps(fn)
```

```
    def wrapper(*args, **kwargs):
        try:
            return fn(*args, **kwargs)
        except KeyboardInterrupt:
            log.info("%s прервано пользователем (Ctrl+C)", fn.
__name__)
            return
        except Exception:
            log.exception("Сбой в %s", fn.__name__)
            if not args or not getattr(args[0], "_root_alive",
lambda: True)():
                return  # окно уже закрыто — исключение уже в
логе
            try:
                messagebox.showerror(
                    "Ошибка",
                    f"Сбой в «{fn.__name__}». Подробности — в app
.log",
                )
            except Exception:
                pass
    return wrapper


class App:
    def __init__(self, root):
        self.root = root
```

```
        root.title("Коррекция хроматических аберраций")
        self.fixed_frame = None
        self._busy = False
        self._pending = None
        self._last_stem = "result"

        bar = tk.Frame(root)
        bar.pack(fill=tk.X, padx=4, pady=4)
        self._buttons = [
            tk.Button(bar, text="Загрузить снимок", command=self.
load_photo)
        ]
        self._buttons[0].pack(side=tk.LEFT, padx=3)

        self.status = tk.Label(
            root, text="Загрузите снимок — коррекция выполнится
автоматически.\n"
                       "Результат до«после/» откроется в
системном просмотрщике.",
            anchor="w", justify=tk.LEFT,
        )
        self.status.pack(fill=tk.X, padx=6, pady=4)

    # --- фоновое выполнение
-----------------------------------------

    def _root_alive(self):
```

```
        try:
            return bool(self.root.winfo_exists())
        except tk.TclError:
            return False

    def _run_job(self, job):
        """Запускает job() в фоновом потоке; результат
помещается в
        self._pending и забирается главным потоком поллингом."""
        if self._busy:
            return
        self._busy = True
        self._pending = None
        for b in self._buttons:
            b.config(state=tk.DISABLED)
        self.status.config(text="Обработка… окно отвечает,
подождите пару секунд")

        def worker():
            try:
                self._pending = (True, job())
            except Exception:
                log.exception("Сбой фоновой обработки")
                self._pending = (False, None)

        threading.Thread(target=worker, daemon=True, name="proc")
.start()
```

```
        self.root.after(100, self._poll_pending)

    def _poll_pending(self):
        if self._pending is None:
            self.root.after(100, self._poll_pending)
            return
        ok, res = self._pending
        self._pending = None
        self._busy = False
        for b in self._buttons:
            b.config(state=tk.NORMAL)
        if ok:
            try:
                self._display(*res)
            except Exception:
                log.exception("Сбой при показе результата — "
                              "файлы уже сохранены на диске")
        else:
            messagebox.showerror(
                "Ошибка", "Сбой при обработке. Подробности — в
app.log")

    # --- действие единственная( кнопка)
----------------------------------

    @log_errors
    def load_photo(self):
```

```
        frame = self._open_image()
        if frame is None:
            return
        stem = self._last_stem
        self._run_job(lambda: self._blind_job(frame, stem))

    # --- фоновая задача без UI, пишет файлы сама ------------------------

    def _blind_job(self, frame, prefix):
        t_start = time.perf_counter()
        center = (frame.shape[1] / 2, frame.shape[0] / 2)
        log.info("Слепая коррекция %dx%d", frame.shape[1], frame.shape[0])
        cur = frame
        acc_r = acc_b = 0.0
        n_edges = 0
        for i in range(3):  # итеративное уточнение до сходимости
            t0 = time.perf_counter()
            r, dr_red, dr_blue = estimate_displacements_blind(cur, center)
            n_edges = len(r)
            if n_edges == 0:
                raise ValueError("Не найдено достаточно краёв")
            cr = fit_radial(r, dr_red, degree=1)
            cb = fit_radial(r, dr_blue, degree=1)
            rmax = r.max()
```

```
            d_r = radial_model(cr, rmax)
            d_b = radial_model(cb, rmax)
            log.info("итерация %d: оценка по %d краям за %.1cf "
                     "сдвиг( R %+.2f, B %+.2f)",
                     i + 1, n_edges, time.perf_counter() - t0,
d_r, d_b)
            if max(abs(d_r), abs(d_b)) < 0.5:
                break
            acc_r += d_r
            acc_b += d_b
            t0 = time.perf_counter()
            cur = correct(cur, center, cr, cb)
            log.info("итерация %d: варп за %.1cf", i + 1,
                     time.perf_counter() - t0)
        log.info("Слепая коррекция завершена за %.1cf",
                 time.perf_counter() - t_start)
        unchanged = cur is frame
        status = (
            f"Автокоррекция по {n_edges} краям: сдвиг на краю
кадра "
            f"R {acc_r:+.1f}px, B {acc_b:+.1f}px устранён
остаток( <0.7px)."
            if not unchanged else "Снимок не требовал коррекции (
CA < 0.5px)."
        )
        info = ("Латеральная хроматическая аберрация меньше 0.5px
 _ "
```

```
                "снимок не требует коррекции.") if unchanged else
 None
      diff_path = self._write_outputs(frame, cur, prefix)
      return (frame, cur, diff_path, status, info)

  def _write_outputs(self, before, after, prefix):
      """Сохраняет в results/ кадры до и после коррекции, а
также
      парное изображение для сравнения увеличенные( углы)."""
      t0 = time.perf_counter()
      out = os.path.join("results", prefix)
      os.makedirs("results", exist_ok=True)
      cv2.imwrite(f"{out}_before.png", before)
      cv2.imwrite(f"{out}_after.png", after)
      zb, za = self._corner_zoom(before, after)
      cv2.imwrite(f"{out}_diff.png", np.hstack([zb, za]))
      log.info("PNGфайлы- записаны за %.1cf", time.perf_counter
() - t0)
      return f"{out}_diff.png"

  # --- вывод только( главный поток)
-----------------------------------

  def _display(self, before, after, diff_path, status, info):
      self.fixed_frame = after
      stem = os.path.basename(diff_path).replace("_diff.png", "
")
```

```
        out_dir = os.path.dirname(os.path.abspath(diff_path))
        self.status.config(
            text=f"{status}\Результатыn в папке {out_dir}: "
                 f"{stem}_before.png, {stem}_after.png, {stem}
_diff.png")
        # Результат открывается в системном просмотрщике
изображений
        try:
            subprocess.run(["open", diff_path], check=False)
            log.info("_display: открыт в просмотрщике %s",
diff_path)
        except OSError:
            log.warning("_display: не удалось открыть
просмотрщик")
        if info:
            messagebox.showinfo("CA не обнаружена", info)
        self.root.lift()

    def _corner_zoom(self, before, after):
        """Увеличенные кропы наиболее удалённого от центра угла
кадра,
        где хроматическая аберрация максимальна."""
        h, w = before.shape[:2]
        cx, cy = w / 2, h / 2
        corner = max(
            [(0, 0), (w, 0), (0, h), (w, h)],
            key=lambda p: (p[0] - cx) ** 2 + (p[1] - cy) ** 2,
```

```
        )
        side = min(h, w) // 3
        x0 = w - side if corner[0] >= cx else 0
        y0 = h - side if corner[1] >= cy else 0
        sl = (slice(y0, y0 + side), slice(x0, x0 + side))

        def zoom(img):
            return cv2.resize(img[sl], (THUMB_W, THUMB_W),
                              interpolation=cv2.INTER_LINEAR)

        return zoom(before), zoom(after)

    # --- утилиты
-------------------------------------------------------------

    def _open_image(self):
        path = filedialog.askopenfilename()
        if not path:
            return None
        frame = cv2.imread(path, cv2.IMREAD_COLOR)
        if frame is None:
            log.warning("Не удалось прочитать изображение: %s",
path)
            messagebox.showerror("Ошибка", "Не удалось открыть
изображение")
        else:
            stem = os.path.splitext(os.path.basename(path))[0]
```

```
            self._last_stem = stem or "result"
            log.info("Загружено %s (%dx%d)", path,
                     frame.shape[1], frame.shape[0])
        return frame


if __name__ == "__main__":
    try:
        log.info("Старт приложения")
        root = tk.Tk()
        App(root)
        root.mainloop()
        log.info("Успешный выход")
    except KeyboardInterrupt:
        log.info("Выход по Ctrl+C")
        print("\Успешныйn выход")
    except Exception:
        log.exception("Критический сбой приложения")
        raise
```